PRAXISHANDBUCH

Burp Suite

Das Praxishandbuch

Webanwendungen systematisch prüfen – vom Proxy bis zum automatisierten Scan



JEDE ANFRAGE SEHEN. GEZIELT VERÄNDERN. NUR IM EIGENEN SCOPE.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Burp Suite
Das Praxishandbuch

Webanwendungen systematisch prüfen – vom Proxy bis zum automatisierten Scan

Von Proxy und Repeater über Intruder bis zu Scope, Scanner und Erweiterungen.
Mit geführten Übungen gegen DVWA, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist Burp Suite? 7

1.1 Einordnung: die Anwendungsschicht statt des Netzwerks 7

1.2 Community vs. Professional 7

1.3 Die Oberfläche im Überblick 8

1.4 Dual-Use wie jedes Werkzeug dieser Reihe 8

Zusammenfassung 8

Kapitel 2: Grundlagen – HTTP-Proxy-Prinzip und TLS-Interception 9

2.1 Das Proxy-Prinzip 9

2.2 Das Problem bei HTTPS 9

2.3 Die Lösung: das Burp-CA-Zertifikat 9

2.4 Warum das bei unautorisiertem Einsatz besonders kritisch ist 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Installation und Start 11

3.2 Browser-Proxy einrichten 11

3.3 Das Burp-CA-Zertifikat installieren 11

3.4 DVWA als Zielanwendung 12

Zusammenfassung 12

Kapitel 4: Proxy – Intercept und HTTP History 13

4.1 Intercept: Anfragen vor dem Senden anhalten 13

4.2 HTTP History: der vollständige Mitschnitt 13

4.3 Gezielt filtern 14

Zusammenfassung 14

Kapitel 5: Repeater – gezieltes Wiederholen und Verändern 15

5.1 Vom Proxy zum Repeater 15

5.2 Eine SQL-Injection systematisch testen 15

5.3 Antworten im Verlauf vergleichen 16

5.4 Header und Methode anpassen 16

Zusammenfassung . . . 16

Kapitel 6: Intruder – automatisierte Variation von Anfragen . . . 17

6.1 Positionen markieren . . . 17
6.2 Die vier Angriffstypen . . . 17
6.3 Payload-Listen konfigurieren . . . 18
6.4 Ergebnisse auswerten . . . 18
Zusammenfassung . . . 18

Kapitel 7: Decoder und Comparer . . . 20

7.1 Der Decoder . . . 20
7.2 Eigene Kodierungsketten . . . 21
7.3 Der Comparer . . . 21
Zusammenfassung . . . 21

Kapitel 8: Target, Scope und Scanner . . . 22

8.1 Scope definieren . . . 22
8.2 Die Site-Map . . . 22
8.3 Der automatisierte Scanner (Professional Edition) . . . 22
8.4 Grenzen des automatisierten Scanners . . . 23
Zusammenfassung . . . 23

Kapitel 9: Erweiterungen – der BApp Store . . . 24

9.1 Was ist der BApp Store? . . . 24
9.2 Beispiele verbreiteter Erweiterungen . . . 24
9.3 Das Vertrauensrisiko von Erweiterungen . . . 24
9.4 Wann lohnt sich eine Erweiterung? . . . 25
Zusammenfassung . . . 25

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 26

10.1 Laboraufbau . . . 26
10.2 Checkliste: sichere Laborumgebung . . . 26
10.3 Übungen . . . 26
Zusammenfassung . . . 28

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 29

11.1 Häufige Probleme . . . 29
11.2 Spickzettel . . . 29
11.3 Glossar . . . 30

11.4 Weiterführende Ressourcen 30
Zusammenfassung 30

Verzeichnis der Befehlsbeispiele 31

Index 32

Vorwort

`Burp Suite` ist das Standardwerkzeug für die Prüfung von Webanwendungen und wurde im Ethical-Hacking-Handbuch dieser Reihe bereits kurz als Proxy vorgestellt (Kapitel 5.4). Während die bisherigen Bände vor allem Netzwerke, Pakete und Passwort-Hashes behandeln, verschiebt sich der Fokus hier auf die Anwendungsschicht: HTTP-Anfragen und -Antworten, die zwischen Browser und Webanwendung hin- und hergehen – genau dort, wo SQL-Injection, XSS und Authentifizierungsfehler aus Kapitel 6 des Ethical-Hacking-Handbuchs tatsächlich sichtbar werden.

Dieses Handbuch vertieft, was dort nur angerissen wurde: den vollständigen Werkzeugkasten von Burp Suite – Proxy, Repeater, Intruder, Decoder, Comparer, Scope und Scanner – und zeigt, wie sich die bereits bekannten Web-Schwachstellen damit systematisch statt zufällig finden lassen.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie in den anderen Büchern dieser Reihe gilt ohne Ausnahme: Burp Suite darf **ausschließlich** gegen Webanwendungen eingesetzt werden, zu deren Test du ausdrücklich berechtigt bist – eigene Testumgebungen wie DVWA, ein schriftlich autorisierter Auftrag, oder ein Programm, das Tests ausdrücklich erlaubt (z. B. ein Bug-Bounty-Programm im definierten Scope). Das unbefugte Abfangen oder Verändern fremden Datenverkehrs ist nach §§ 202a/202b StGB strafbar.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das Ethical-Hacking-Handbuch kennst und die dortige, kurze Burp-Suite-Einführung vertiefen möchtest,
- Webanwendungen systematisch statt nur durch manuelles Ausprobieren im Browser prüfen willst,
- verstehen willst, wie sich SQL-Injection, XSS und Authentifizierungsfehler technisch über HTTP-Anfragen aufspüren lassen,
- dich auf Zertifizierungen vorbereitest, in denen Web-Application-Testing eine Rolle spielt (eJPT, OSCP, OSWE).

Aufbau

Kapitel	Thema
1	Was ist Burp Suite? Einordnung, Community vs. Professional
2	Grundlagen: HTTP-Proxy-Prinzip und TLS-Interception
3	Installation und Laborumgebung
4	Proxy: Intercept und HTTP History
5	Repeater: gezieltes Wiederholen und Verändern einzelner Anfragen
6	Intruder: automatisierte Variation von Anfragen
7	Decoder und Comparer
8	Target, Scope und Scanner
9	Erweiterungen: der BApp Store
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Die SQL-Injection- und XSS-Beispiele aus dem Ethical-Hacking-Handbuch (Kapitel 6) lassen sich mit den Werkzeugen aus diesem Buch – besonders Repeater und Intruder – deutlich systematischer nachvollziehen als durch wiederholtes manuelles Eintippen im Browser.

Kapitel 1: Was ist Burp Suite?

Lernziele

- Du kannst Burp Suite als Web-Proxy von netzwerkorientierten Werkzeugen dieser Reihe abgrenzen.
- Du kennst den Unterschied zwischen Community und Professional Edition.
- Du verstehst den grundsätzlichen Aufbau der Burp-Suite-Oberfläche.

1.1 Einordnung: die Anwendungsschicht statt des Netzwerks

Wireshark macht Netzwerkverkehr auf allen Schichten sichtbar (Wireshark-Handbuch, Kapitel 2.3), Nmap und hping3 erkunden Netzwerke und Ports. Burp Suite konzentriert sich dagegen ausschließlich auf eine einzige Schicht: **HTTP/HTTPS zwischen Browser und Webanwendung**. Es schaltet sich als **Proxy** dazwischen und macht jede Anfrage und jede Antwort sichtbar, veränderbar und wiederholbar.

Werkzeug	Fokus
Wireshark	gesamter Netzwerkverkehr, alle Protokolle und Schichten
Nmap, hping3	Netzwerk- und Porterkundung
Burp Suite	ausschließlich HTTP/HTTPS zwischen Browser und Webanwendung

Gut zu wissen

Burp Suite wird von der Firma **PortSwigger** entwickelt, benannt nach dem „Burping" (Aufstoßen) eines Servers, der plötzlich mit unerwarteten Anfragen konfrontiert wird. Die **Web Security Academy** desselben Herstellers ist eine kostenlose, sehr detaillierte Lernplattform zu Web-Schwachstellen und wird in Kapitel 11 als Ressource empfohlen.

1.2 Community vs. Professional

Funktion	Community Edition	Professional Edition
Proxy, Repeater, Decoder, Comparer	vollständig enthalten	vollständig enthalten
Intruder	enthalten, aber künstlich verlangsamt	voll geschwindigkeit
Scanner (automatisierte Schwachstellensuche)	nicht enthalten	enthalten
Erweiterungen (BApp Store)	eingeschränkt nutzbar	vollständig nutzbar

Dieses Handbuch arbeitet durchgehend mit der **kostenlosen Community Edition** – alle Kernkonzepte (Proxy, Repeater, Intruder-Prinzip, Decoder, Comparer) sind dort vollständig vorhanden, lediglich der automatisierte Scanner (Kapitel 8) bleibt der kostenpflichtigen Variante vorbehalten.

1.3 Die Oberfläche im Überblick

Beispiel 1.1: Die wichtigsten Burp-Suite-Tabs (Konzept)

```
Proxy      -> Anfragen abfangen und mitschneiden (Kapitel 4)
Repeater   -> einzelne Anfragen gezielt wiederholen und verändern (Kapitel 5)
Intruder   -> automatisierte Variation von Anfragen (Kapitel 6)
Decoder    -> Kodierungen umwandeln (Kapitel 7)
Comparer   -> zwei Antworten gegenüberstellen (Kapitel 7)
Target     -> Scope und Struktur einer Anwendung (Kapitel 8)
```

Jede in Proxy oder Repeater untersuchte Anfrage lässt sich per Rechtsklick direkt an ein anderes Werkzeug (z. B. „Send to Intruder") weiterreichen – dieser durchgängige Arbeitsablauf ist eines der zentralen Stärken von Burp Suite.

1.4 Dual-Use wie jedes Werkzeug dieser Reihe

LEGAL – nur mit Auftrag/Einwilligung

Auch Burp Suite hat keine Funktion, die „nur für legale Zwecke" reserviert ist. Derselbe Proxy, der einen autorisierten Pentest unterstützt, kann ebenso fremden, nicht für den eigenen Zugriff bestimmten Datenverkehr abfangen. Der einzige Unterschied ist – wie im gesamten Ethical-Hacking-Handbuch – die **Autorisierung**: eigene Testumgebung, autorisierter Auftrag, oder ein Programm mit ausdrücklich erlaubtem Scope (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- Burp Suite konzentriert sich ausschließlich auf HTTP/HTTPS zwischen Browser und Webanwendung.
- Die kostenlose Community Edition deckt Proxy, Repeater, Intruder-Prinzip, Decoder und Comparer vollständig ab; der automatisierte Scanner bleibt der Professional Edition vorbehalten.
- Proxy, Repeater, Intruder, Decoder, Comparer und Target bilden die zentralen Werkzeuge, die sich nahtlos ineinander überführen lassen.
- Wie jedes Werkzeug dieser Reihe entscheidet ausschließlich die Autorisierung über Legalität, nicht die Technik.

Kapitel 2: Grundlagen – HTTP-Proxy-Prinzip und TLS-Interception

Lernziele

- Du verstehst, wie sich Burp Suite technisch zwischen Browser und Server schaltet.
- Du kennst den Zweck des Burp-CA-Zertifikats bei verschlüsseltem Verkehr.
- Du kannst erklären, warum diese Technik bei unautorisiertem Einsatz besonders kritisch ist.

2.1 Das Proxy-Prinzip

Ein **Proxy** sitzt zwischen zwei Kommunikationspartnern und leitet deren Datenverkehr weiter, kann ihn dabei aber auch einsehen und verändern.

Beispiel 2.1: Normale Verbindung vs. Verbindung über einen Proxy (Konzept)

```
Ohne Proxy:  Browser  <----------------------->  Webserver
Mit Proxy:   Browser  <--->  Burp Suite  <--->  Webserver
```

Für unverschlüsseltes HTTP ist das unkompliziert: Burp Suite sieht und kann jede Anfrage und Antwort im Klartext bearbeiten, genau wie in Wireshark sichtbar (Wireshark-Handbuch, Kapitel 7.2), nur mit der zusätzlichen Möglichkeit, aktiv einzugreifen, statt nur mitzulesen.

2.2 Das Problem bei HTTPS

Bei **HTTPS** ist die Verbindung Ende-zu-Ende verschlüsselt. Ohne Weiteres könnte ein dazwischengeschalteter Proxy den Inhalt nicht lesen – genau das, was TLS eigentlich verhindern soll (vergleiche Wireshark-Handbuch, Kapitel 7.3).

Beispiel 2.2: Das Grundproblem (Konzept)

```
Browser <--TLS verschlüsselt--> Burp Suite <--TLS verschlüsselt--> Server
-> Ohne weitere Maßnahme könnte Burp Suite den Inhalt nicht einsehen
```

2.3 Die Lösung: das Burp-CA-Zertifikat

Burp Suite löst das, indem es **zwei getrennte TLS-Verbindungen** aufbaut: eine zum Browser (mit einem von Burp selbst ausgestellten Zertifikat) und eine zum eigentlichen Server (mit dessen echtem Zertifikat). Damit der Browser dem von Burp ausgestellten Zertifikat vertraut, muss das **Burp-CA-Zertifikat** einmalig als vertrauenswürdig installiert werden.

Beispiel 2.3: Zwei getrennte TLS-Verbindungen (Konzept)

```
Browser <--TLS mit Burp-Zertifikat--> Burp Suite <--TLS mit echtem Zertifikat--> Server
```

WARNUNG – Sicherheitsrisiko

Das Installieren eines Burp-CA-Zertifikats als vertrauenswürdig ist technisch identisch mit dem, was bei einem sogenannten **Man-in-the-Middle-Angriff** auf TLS-Verkehr geschieht – der einzige Unterschied ist, dass du es **bewusst und für dein eigenes Gerät** einrichtest. Installiere das Burp-CA-Zertifikat ausschließlich in einem dafür vorgesehenen Testprofil oder Browser (Kapitel 3), nie systemweit auf einem produktiv genutzten Gerät.

2.4 Warum das bei unautorisiertem Einsatz besonders kritisch ist

ILLEGAL – niemals ohne Auftrag

Dieselbe Technik, die Burp Suite nutzt, um deinen **eigenen** verschlüsselten Verkehr einsehbar zu machen, würde gegen einen fremden Nutzer oder ein fremdes Gerät eingesetzt, dessen vollständigen, eigentlich geschützten Datenverkehr offenlegen – ein deutlich schwerwiegenderer Eingriff als ein einfacher Netzwerkmitschnitt (Wireshark-Handbuch, Kapitel 1.4). Das Installieren eines fremden Zertifikats auf einem Gerät, das dir nicht gehört, oder das Umleiten fremden Datenverkehrs über deinen Proxy ohne Einwilligung ist strafbar.

Zusammenfassung

Zusammenfassung

- Ein Proxy sitzt zwischen zwei Kommunikationspartnern und kann deren Datenverkehr einsehen und verändern.
- Bei HTTPS baut Burp Suite zwei getrennte TLS-Verbindungen auf und braucht dafür ein eigenes, vom Browser als vertrauenswürdig akzeptiertes Zertifikat.
- Das Burp-CA-Zertifikat gehört ausschließlich in ein dafür vorgesehenes Testprofil, nie systemweit auf ein produktiv genutztes Gerät.
- Dieselbe Technik gegen fremde Geräte oder fremden Verkehr eingesetzt, ist ein schwerwiegender, strafbarer Eingriff.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du startest Burp Suite Community Edition unter Kali.
- Du richtest Browser-Proxy und Burp-CA-Zertifikat in einem eigenen Testprofil ein.
- Du nutzt DVWA als Zielanwendung für alle folgenden Kapitel.

3.1 Installation und Start

Beispiel 3.1: Burp Suite unter Kali starten

```
# Kali Linux: bereits vorinstalliert
$ burpsuite
```

Unter anderen Distributionen lässt sich die Community Edition kostenlos von der offiziellen PortSwigger-Projektseite herunterladen.

3.2 Browser-Proxy einrichten

Beispiel 3.2: Proxy-Listener in Burp Suite prüfen

```
Burp Suite -> Reiter "Proxy" -> "Options" -> Standard-Listener: 127.0.0.1:8080
```

Beispiel 3.3: Browser-Proxy auf Burp Suite umstellen (Konzept, Firefox)

```
Firefox-Einstellungen -> Netzwerk -> Manuelle Proxy-Konfiguration
HTTP-Proxy: 127.0.0.1   Port: 8080
```

TIPP – Profi-Trick

Richte für diese Übungen ein **eigenes Firefox-Profil** ein (`firefox -P`), das ausschließlich für Burp-Suite-Sitzungen genutzt wird. So bleibt dein normales Surfprofil unberührt von der Proxy-Umstellung und dem installierten Burp-Zertifikat.

3.3 Das Burp-CA-Zertifikat installieren

Beispiel 3.4: Zertifikat herunterladen und installieren (Konzept)

```
1. Browser-Proxy auf Burp Suite umstellen (Beispiel 3.3)
2. Im Browser http://burpsuite aufrufen
3. "CA Certificate" herunterladen
4. Im Testprofil des Browsers als vertrauenswürdige Zertifizierungsstelle importieren
```

WARNUNG – Sicherheitsrisiko

Importiere das Zertifikat ausschließlich in das in Beispiel 3.2 angelegte, separate Testprofil – niemals in dein alltäglich genutztes Browserprofil (Kapitel 2.3).

3.4 DVWA als Zielanwendung

Dieses Buch nutzt dieselbe Zielanwendung wie das Ethical-Hacking-Handbuch (Kapitel 6 und 9): **DVWA** (Damn Vulnerable Web Application), per Docker in der Kali-VM betrieben.

Beispiel 3.5: DVWA starten (Wiederholung aus dem Ethical-Hacking-Handbuch)

```
$ sudo docker run --rm -it -p 127.0.0.1:80:80 vulnerables/web-dvwa
```

LEGAL – nur mit Auftrag/Einwilligung

Binde DVWA ausschließlich an `127.0.0.1` und greife ausschließlich über dein Testprofil darauf zu. Dieselbe Netzwerkisolation wie in den anderen Büchern dieser Reihe gilt auch hier uneingeschränkt (Ethical-Hacking-Handbuch, Kapitel 9).

Zusammenfassung

Zusammenfassung

- Burp Suite Community Edition ist unter Kali vorinstalliert, sonst kostenlos von PortSwigger verfügbar.
- Ein eigenes Browserprofil für Burp-Suite-Sitzungen hält Proxy-Einstellungen und Zertifikat vom normalen Surfprofil getrennt.
- Das Burp-CA-Zertifikat wird einmalig heruntergeladen und ausschließlich im Testprofil als vertrauenswürdig importiert.
- DVWA, dieselbe Zielanwendung wie im Ethical-Hacking-Handbuch, dient als Grundlage für alle folgenden Kapitel.

Kapitel 4: Proxy – Intercept und HTTP History

Lernziele

- Du fängst eine einzelne Anfrage gezielt ab und veränderst sie, bevor sie weitergeleitet wird.
- Du nutzt die HTTP History, um vergangene Anfragen nachträglich zu durchsuchen.
- Du filterst die History gezielt nach relevantem Verkehr.

4.1 Intercept: Anfragen vor dem Senden anhalten

Beispiel 4.1: Intercept aktivieren und eine Anfrage abfangen

```
Burp Suite -> Reiter "Proxy" -> "Intercept" -> "Intercept is on"
# Im Browser (Testprofil) eine beliebige Seite von DVWA aufrufen
# Die Anfrage erscheint angehalten im Intercept-Fenster
```

Beispiel 4.2: Eine abgefangene Anfrage anzeigen und bearbeiten

```
GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: localhost
Cookie: PHPSESSID=abc123; security=low
```

Im Intercept-Fenster lässt sich jedes Feld der Anfrage direkt bearbeiten – Parameter, Header, sogar die HTTP-Methode – bevor sie mit „Forward" tatsächlich an den Server gesendet wird. „Drop" verwirft die Anfrage vollständig, ohne sie zu senden.

TIPP – Profi-Trick

Intercept dauerhaft aktiviert zu lassen, wird bei normalem Browsen schnell mühsam, weil jede einzelne Anfrage manuell weitergeleitet werden muss. Für die meisten Aufgaben ist es praktischer, Intercept nur kurz für die eine, relevante Anfrage einzuschalten und danach wieder zu deaktivieren.

4.2 HTTP History: der vollständige Mitschnitt

Beispiel 4.3: HTTP History durchsuchen

```
Burp Suite -> Reiter "Proxy" -> "HTTP history"
```

Während Intercept jede Anfrage einzeln anhält, zeichnet die **HTTP History** im Hintergrund **alle** Anfragen und Antworten auf, unabhängig davon, ob Intercept gerade aktiv ist – vergleichbar mit der Paketliste in Wireshark (Wireshark-Handbuch, Kapitel 4.2), nur auf der Ebene einzelner HTTP-Transaktionen statt einzelner Pakete.

Beispiel 4.4: Einen Eintrag der History im Detail ansehen

```
Klick auf eine Zeile -> Reiter "Request" zeigt die gesendete Anfrage
                     -> Reiter "Response" zeigt die empfangene Antwort
```

4.3 Gezielt filtern

Beispiel 4.5: HTTP History nach Methode und Statuscode filtern

```
Filter-Leiste oberhalb der History -> "Show only: POST requests"
Filter-Leiste -> Status code: 200
```

> **Gut zu wissen**
>
> Ein besonders nützlicher History-Filter ist die Suche nach bestimmten Parameternamen (z. B. `id=`), um gezielt alle Anfragen zu finden, die einen bestimmten, potenziell verwundbaren Parameter enthalten – ein guter erster Schritt, bevor du eine konkrete Anfrage zur weiteren Untersuchung an den Repeater sendest (Kapitel 5).

Zusammenfassung

> **Zusammenfassung**
>
> - Intercept hält einzelne Anfragen an, bevor sie gesendet werden, und erlaubt direkte Bearbeitung vor dem Weiterleiten.
> - Die HTTP History zeichnet durchgehend alle Anfragen und Antworten auf, unabhängig vom Intercept-Status.
> - Filter nach Methode, Statuscode oder Parameternamen helfen, in einem großen Mitschnitt gezielt relevante Anfragen zu finden.
> - Ein typischer Arbeitsablauf führt von der History über eine interessante Anfrage direkt zum Repeater.

Kapitel 5: Repeater – gezieltes Wiederholen und Verändern

Lernziele

- Du sendest eine einzelne Anfrage gezielt wiederholt an den Server, mit jeweils eigenen Änderungen.
- Du nutzt den Repeater, um eine vermutete SQL-Injection systematisch zu testen.
- Du vergleichst mehrere Antworten im Repeater-Verlauf.

5.1 Vom Proxy zum Repeater

Beispiel 5.1: Eine Anfrage aus der History an den Repeater senden

```
HTTP History -> Rechtsklick auf eine Anfrage -> "Send to Repeater"
```

Der **Repeater** nimmt eine einzelne Anfrage entgegen und erlaubt, sie beliebig oft mit jeweils eigenen Änderungen erneut zu senden – ohne jedes Mal den ursprünglichen Ablauf im Browser wiederholen zu müssen.

5.2 Eine SQL-Injection systematisch testen

Beispiel 5.2: Ursprüngliche Anfrage im Repeater

```
GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: localhost
Cookie: PHPSESSID=abc123; security=low
```

Beispiel 5.3: Denselben Parameter systematisch verändern und senden

```
Veränderung 1: id=1'               -> Antwort auf SQL-Fehlermeldung prüfen
Veränderung 2: id=1' OR '1'='1     -> mit Ethical-Hacking-Handbuch,
                                      Kapitel 6.1 vergleichen
Veränderung 3: id=1' UNION SELECT user, password FROM users -- -
```

Jede dieser drei Veränderungen lässt sich im Repeater durch einfaches Überschreiben des Parameterwerts und erneutes Klicken auf „Send" nacheinander testen, ohne jedes Mal zur Webanwendung zurückzuwechseln – genau der systematische Ablauf, der im Ethical-Hacking-Handbuch (Kapitel 6.1) nur im Browser beschrieben wird.

ILLEGAL – niemals ohne Auftrag

Teste diese Veränderungen ausschließlich gegen DVWA oder eine vergleichbare, dafür vorgesehene Laborumgebung (Kapitel 3.4). Gegen ein reales, fremdes System ist bereits der erste Testwert `id=1'` eine unautorisierte, aktive Manipulation.

5.3 Antworten im Verlauf vergleichen

Beispiel 5.4: Der Repeater-Verlauf

```
Jede gesendete Variante erscheint als eigener, nummerierter Eintrag im
Verlauf des Repeater-Tabs und lässt sich jederzeit erneut aufrufen.
```

TIPP – Profi-Trick

Achte beim Vergleich der Antworten nicht nur auf offensichtliche Fehlermeldungen, sondern auch auf **subtile Unterschiede**: eine leicht veränderte Antwortlänge, ein zusätzliches Datenfeld, oder ein abweichender Statuscode können auf eine erfolgreiche, aber nicht offensichtlich sichtbare Manipulation hindeuten.

5.4 Header und Methode anpassen

Beispiel 5.5: HTTP-Methode direkt im Repeater ändern

```
GET  ->  POST
```

Der Repeater erlaubt, nicht nur Parameterwerte, sondern auch Header und sogar die HTTP-Methode selbst zu verändern – nützlich, um zu prüfen, ob eine Anwendung dieselbe Funktion über mehrere Methoden anbietet und dabei unterschiedlich streng prüft.

Zusammenfassung

Zusammenfassung

- Der Repeater sendet eine einzelne Anfrage beliebig oft mit jeweils eigenen Änderungen erneut.
- Systematisches Testen einer SQL-Injection im Repeater ersetzt das wiederholte manuelle Ausprobieren im Browser.
- Der Verlauf hält jede gesendete Variante nachvollziehbar fest, einschließlich der jeweiligen Antwort.
- Neben Parametern lassen sich auch Header und die HTTP-Methode direkt im Repeater anpassen.

Kapitel 6: Intruder – automatisierte Variation von Anfragen

Lernziele

- Du kennst die vier Angriffstypen des Intruders und ihre jeweilige Einsatzsituation.
- Du setzt Positionsmarker gezielt in einer Anfrage.
- Du verstehst, warum die Community Edition den Intruder absichtlich verlangsamt.

ILLEGAL – niemals ohne Auftrag

Dieses Kapitel behandelt automatisiertes, massenhaftes Durchprobieren von Anfragevarianten – vergleichbar mit Hydra (Ethical-Hacking-Handbuch, Kapitel 6.4) oder den Angriffsmodi von John the Ripper und Hashcat, nur auf HTTP-Ebene. Ausschließlich gegen DVWA oder eine vergleichbare, dafür vorgesehene Laborumgebung einsetzen.

6.1 Positionen markieren

Beispiel 6.1: Eine Anfrage an den Intruder senden und Position markieren

```
HTTP History -> Rechtsklick -> "Send to Intruder"
Im Intruder-Tab: den zu variierenden Parameterwert markieren -> "Add §"
```

Beispiel 6.2: Eine markierte Anfrage (Konzept)

```
GET /vulnerabilities/brute/?username=admin&password=§password§&Login=Login HTTP/1.1
```

Die §-Zeichen markieren die Position, die der Intruder bei jedem Durchlauf durch einen neuen Kandidaten aus einer Payload-Liste ersetzt.

6.2 Die vier Angriffstypen

Angriffstyp	Positionen	Payload-Listen	Verhalten
Sniper	eine oder mehrere	eine	testet jede Position einzeln nacheinander mit derselben Liste
Battering Ram	mehrere	eine	setzt denselben Kandidaten gleichzeitig in alle markierten Positionen
Pitchfork	mehrere	mehrere, parallel	Position 1 mit Liste 1, Position 2 mit Liste 2, synchron fortschreitend
Cluster Bomb	mehrere	mehrere, kombiniert	testet alle Kombinationen aus allen Listen gegeneinander

Beispiel 6.3: Cluster Bomb gegen Benutzername und Passwort (Konzept)

```
Position 1 (username): admin, administrator
Position 2 (password): password, admin123, letmein
-> 2 x 3 = 6 Kombinationen werden automatisch getestet
```

Dieses Grundprinzip entspricht exakt dem im Prompting-Handbuch dieser Reihe beschriebenen Cluster-Bomb-Beispiel (dort als Analogie für Sicherheitstests erwähnt) – hier in seiner tatsächlichen, technischen Anwendung gegen ein Web-Login.

6.3 Payload-Listen konfigurieren

Beispiel 6.4: Eine einfache Payload-Liste einsetzen

```
Intruder -> Reiter "Payloads" -> Payload-Typ "Simple list"
Liste: password, admin123, letmein, test123
```

> **Gut zu wissen**
>
> Für Payload-Listen lassen sich dieselben Wortlisten wiederverwenden, die bereits in anderen Büchern dieser Reihe zum Einsatz kommen (z. B. `rockyou.txt`, John-the-Ripper-Handbuch, Kapitel 3.3) – das Grundprinzip „Wortliste gegen ein Ziel testen" ist werkzeugübergreifend identisch, nur die Zielform (HTTP-Parameter statt Passwort-Hash) unterscheidet sich.

6.4 Ergebnisse auswerten

Beispiel 6.5: Ergebnisse nach Antwortlänge sortieren

```
Intruder-Ergebnistabelle -> Spalte "Length" anklicken, um zu sortieren
```

Ein erfolgreicher Login-Versuch unterscheidet sich oft durch eine **abweichende Antwortlänge** oder einen anderen Statuscode von den fehlgeschlagenen Versuchen – ein Sortieren nach diesen Spalten macht Ausreißer schnell sichtbar, ohne jede einzelne Antwort manuell lesen zu müssen.

> **WARNUNG – Sicherheitsrisiko**
>
> Die **Community Edition** verlangsamt den Intruder absichtlich gegenüber der Professional Edition. Für die Übungen in diesem Buch mit kleinen, selbst erstellten Listen ist das unerheblich; bei großen Wortlisten macht sich der Unterschied in der Laufzeit deutlich bemerkbar.

Zusammenfassung

> **Zusammenfassung**
>
> - Sniper, Battering Ram, Pitchfork und Cluster Bomb sind die vier Angriffstypen des Intruders, mit unterschiedlichem Zusammenspiel von Positionen und Payload-Listen.
> - Positionsmarker (§) legen fest, welche Teile einer Anfrage bei jedem Durchlauf ersetzt werden.
> - Dieselben Wortlisten wie bei John the Ripper oder Hashcat lassen sich als Intruder-Payloads wiederverwenden.
> - Sortieren nach Antwortlänge oder Statuscode macht erfolgreiche Treffer unter vielen Versuchen schnell sichtbar.

Kapitel 7: Decoder und Comparer

Lernziele

- Du dekodierst gängige Web-Kodierungen mit dem Decoder.
- Du erkennst, warum Kodierung nicht mit Verschlüsselung verwechselt werden darf.
- Du vergleichst zwei Antworten gezielt mit dem Comparer.

7.1 Der Decoder

Webanwendungen kodieren Daten aus technischen Gründen häufig um – für die Übertragung in einer URL, für die sichere Einbettung in HTML, oder zur einfachen Verschleierung. Der **Decoder** wandelt gängige Kodierungen in beide Richtungen um.

Beispiel 7.1: Base64 dekodieren

```
Eingabe:  YWRtaW46cGFzc3dvcnQxMjM=
Decoder -> Decode as -> Base64
Ausgabe:  admin:passwort123
```

Dieses Beispiel ist identisch mit dem im Wireshark-Handbuch (Kapitel 7.2) beschriebenen HTTP-Basic-Authentication-Fund – dort über die Kommandozeile dekodiert, hier direkt im Decoder-Tab von Burp Suite.

Beispiel 7.2: URL-Kodierung umwandeln

```
Eingabe:  passwort%21
Decoder -> Decode as -> URL
Ausgabe:  passwort!
```

Kodierung	Typischer Einsatzort
URL-Encoding	Sonderzeichen in URL-Parametern
Base64	binäre oder strukturierte Daten in Textform, z. B. Authentifizierungs-Header
HTML-Entities	Sonderzeichen innerhalb von HTML-Inhalten
Hex	gelegentlich in benutzerdefinierten Anwendungsprotokollen

WARNUNG – Sicherheitsrisiko

Kodierung ist **keine Verschlüsselung** (dieselbe Unterscheidung wie im Wireshark-Handbuch, Kapitel 7.2): Jede der oben genannten Kodierungen lässt sich ohne jeden Schlüssel umkehren. Verwechsle „ich sehe keinen Klartext" nicht mit „die Daten sind geschützt".

7.2 Eigene Kodierungsketten

Beispiel 7.3: Mehrfach kodierte Werte entschlüsseln

```
Eingabe: JTI1MzE=
Schritt 1: Base64-Decode -> %253=
Schritt 2: URL-Decode -> %3=
Schritt 3: URL-Decode -> =
```

Manche Anwendungen kodieren Werte mehrfach hintereinander. Der Decoder erlaubt, mehrere Dekodierungsschritte nacheinander auf dasselbe Ergebnis anzuwenden, bis der eigentliche Klartext sichtbar wird.

7.3 Der Comparer

Der **Comparer** stellt zwei Texte oder zwei HTTP-Antworten nebeneinander und markiert die Unterschiede – nützlich, um subtile Abweichungen zu finden, die beim bloßen Lesen leicht übersehen würden (vergleiche Kapitel 5.3, „subtile Unterschiede" im Repeater-Verlauf).

Beispiel 7.4: Zwei Antworten vergleichen

```
Proxy History -> zwei Einträge auswählen -> Rechtsklick -> "Send to Comparer"
Comparer -> "Words" oder "Bytes" -> Unterschiede werden farblich hervorgehoben
```

> **TIPP – Profi-Trick**
>
> Der Comparer eignet sich besonders gut, um die Antwort auf einen gültigen und einen vermutlich ungültigen Login-Versuch direkt gegenüberzustellen – oft fällt dabei ein winziger, aber entscheidender Unterschied auf (z. B. eine zusätzliche Kopfzeile), der die Grundlage für einen gezielten Intruder-Angriff (Kapitel 6) liefert.

Zusammenfassung

> **Zusammenfassung**
>
> - Der Decoder wandelt gängige Web-Kodierungen (URL, Base64, HTML-Entities, Hex) in beide Richtungen um.
> - Kodierung ist keine Verschlüsselung und lässt sich ohne jeden Schlüssel umkehren.
> - Mehrfach kodierte Werte lassen sich durch wiederholte Dekodierungsschritte schrittweise entschlüsseln.
> - Der Comparer stellt zwei Texte oder Antworten nebeneinander und hebt Unterschiede farblich hervor.

Kapitel 8: Target, Scope und Scanner

Lernziele

- Du definierst einen Scope, der Burp Suite auf die relevante Anwendung begrenzt.
- Du nutzt die Site-Map, um die Struktur einer Anwendung zu überblicken.
- Du verstehst, was der automatisierte Scanner leistet und wo seine Grenzen liegen.

8.1 Scope definieren

Ohne Einschränkung schneidet der Proxy **jeden** Verkehr mit, der über ihn läuft – auch Hintergrundverbindungen des Browsers selbst oder versehentlich aufgerufene, nicht zum Test gehörende Seiten. Der **Scope** begrenzt Burp Suite gezielt auf die relevante Anwendung.

Beispiel 8.1: Scope auf DVWA begrenzen

```
Target -> Site map -> Rechtsklick auf localhost -> "Add to scope"
Proxy -> Options -> "Show only in-scope items" aktivieren
```

Gut zu wissen

Ein sauber definierter Scope entspricht demselben Prinzip wie der vertraglich festgelegte Scope eines Pentests (Ethical-Hacking-Handbuch, Kapitel 1.5) – nur technisch im Werkzeug selbst umgesetzt, statt nur im Vertrag festgehalten.

8.2 Die Site-Map

Beispiel 8.2: Struktur einer Anwendung erkunden

```
Target -> Site map -> Baumstruktur aller bisher besuchten Pfade und Parameter
```

Während du durch DVWA klickst, füllt sich die Site-Map automatisch mit jeder besuchten Seite und jedem verwendeten Parameter – ein wachsender Überblick über die Struktur der Anwendung, vergleichbar mit dem, was Nmap für ein Netzwerk liefert (Nmap-Handbuch, Kapitel 1.3), nur auf Anwendungsebene statt auf Netzwerkebene.

8.3 Der automatisierte Scanner (Professional Edition)

Beispiel 8.3: Einen Scan starten (Konzept, nur Professional Edition)

```
Target -> Site map -> Rechtsklick auf den Scope-Eintrag -> "Scan"
```

Scan-Art	Charakteristik
Passiv	analysiert nur bereits beobachteten Verkehr, sendet keine eigenen Anfragen
Aktiv	sendet gezielt eigene, manipulierte Anfragen, um Schwachstellen auszulösen

WARNUNG – Sicherheitsrisiko

Ein **aktiver Scan** sendet automatisiert sehr viele Testanfragen, darunter auch solche, die Formulare absenden, Daten verändern oder Zustände auslösen können. Das ist in einer Testumgebung wie DVWA unproblematisch, kann aber in einer echten Produktivumgebung unbeabsichtigte Nebenwirkungen haben (etwa versehentlich ausgelöste Bestellungen oder E-Mails). Ein aktiver Scan gegen ein reales Ziel gehört deshalb immer in den vorher vertraglich abgestimmten Scope und sollte mit dem Auftraggeber in Zeitfenster und Umfang abgesprochen sein.

8.4 Grenzen des automatisierten Scanners

Gut zu wissen

Ein automatisierter Scanner findet zuverlässig viele bekannte, strukturell erkennbare Schwachstellenmuster, übersieht aber **logische** Schwachstellen – etwa eine Berechtigungsprüfung, die zwar technisch korrekt aufgebaut ist, aber die falsche Bedingung prüft. Dieselbe Einschränkung gilt für jeden Schwachstellenscanner dieser Reihe (vergleiche Ethical-Hacking-Handbuch, Kapitel 4.4, zu Nmap/OpenVAS): Automatisierung ergänzt die manuelle Prüfung mit Repeater und Intruder, ersetzt sie aber nicht.

Zusammenfassung

Zusammenfassung

- Ein definierter Scope begrenzt Burp Suite auf die tatsächlich relevante Anwendung und blendet Störverkehr aus.
- Die Site-Map baut sich automatisch aus dem beobachteten Verkehr auf und zeigt die Struktur der Anwendung.
- Der automatisierte Scanner (Professional Edition) unterscheidet passive und aktive Scans; aktive Scans können in Produktivumgebungen Nebenwirkungen haben.
- Automatisierte Scans finden strukturelle Muster, übersehen aber logische Schwachstellen und ersetzen nicht die manuelle Prüfung.

Kapitel 9: Erweiterungen – der BApp Store

Lernziele

- Du kennst den Zweck des BApp Store und installierst eine Erweiterung.
- Du verstehst, warum Erweiterungen ein eigenes Vertrauensrisiko mitbringen.
- Du kannst einschätzen, wann sich eine Erweiterung lohnt.

9.1 Was ist der BApp Store?

Der **BApp Store** (Burp App Store) ist ein in Burp Suite integriertes Verzeichnis offizieller und community-entwickelter Erweiterungen, die zusätzliche Funktionen nachrüsten – von zusätzlichen Decoder-Formaten bis zu spezialisierten Scan-Prüfungen.

Beispiel 9.1: BApp Store öffnen und eine Erweiterung installieren

```
Burp Suite -> Reiter "Extensions" -> "BApp Store"
Erweiterung auswählen -> "Install"
```

9.2 Beispiele verbreiteter Erweiterungen

Erweiterung	Zweck
Logger++	erweiterte, durchsuchbare Protokollierung über alle Burp-Werkzeuge hinweg
Autorize	automatisiertes Prüfen von Berechtigungsfehlern (vergleiche Kapitel 8.4, logische Schwachstellen)
JSON Web Tokens	Decoder-Unterstützung speziell für JWT-Token
Turbo Intruder	ein für sehr hohe Geschwindigkeit optimierter, skriptbarer Intruder-Ersatz

Gut zu wissen

Viele Erweiterungen entstehen aus genau den Einschränkungen, die in diesem Buch bereits angesprochen wurden – etwa Turbo Intruder als Antwort auf die absichtliche Verlangsamung des Standard-Intruders in der Community Edition (Kapitel 6.4).

9.3 Das Vertrauensrisiko von Erweiterungen

WARNUNG – Sicherheitsrisiko

Erweiterungen laufen mit denselben Rechten wie Burp Suite selbst und haben vollen Zugriff auf den gesamten durchlaufenden Verkehr, einschließlich sensibler Zugangsdaten. Installiere ausschließlich Erweiterungen aus dem offiziellen BApp Store oder aus vertrauenswürdigen, bekannten Quellen, und informiere dich vor der Installation über den Entwickler und die Bewertungen anderer Nutzer:innen.

9.4 Wann lohnt sich eine Erweiterung?

Situation	Empfehlung
Einzelne, einmalige Übung in diesem Buch	Standardwerkzeuge reichen meist aus
Wiederkehrende, spezialisierte Aufgabe (z. B. JWT-Analyse)	passende Erweiterung aus dem BApp Store prüfen
Sehr große Wortlisten mit dem Intruder	Turbo Intruder als Alternative erwägen
Systematische Berechtigungsprüfungen über viele Endpunkte	Autorize oder vergleichbare Erweiterung erwägen

Zusammenfassung

Zusammenfassung

- Der BApp Store bündelt offizielle und community-entwickelte Erweiterungen direkt in Burp Suite.
- Viele Erweiterungen beheben gezielt Einschränkungen der Standardwerkzeuge, etwa Geschwindigkeit oder spezialisierte Formate.
- Erweiterungen haben vollen Zugriff auf den durchlaufenden Verkehr und bringen ein eigenes Vertrauensrisiko mit.
- Für einmalige Übungen reichen die Standardwerkzeuge meist aus; Erweiterungen lohnen sich vor allem bei wiederkehrenden, spezialisierten Aufgaben.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass Ergebnisse je nach DVWA-Version und Sicherheitsstufe leicht abweichen können.

10.1 Laboraufbau

Dieses Buch nutzt dieselbe Laborumgebung wie das Ethical-Hacking-Handbuch: eine Kali-VM mit Burp Suite Community Edition und DVWA, per Docker betrieben und ausschließlich an `127.0.0.1` gebunden (Kapitel 3).

10.2 Checkliste: sichere Laborumgebung

Checkliste 10.1: Vor jeder Übung

```
[ ] Burp-CA-Zertifikat ist nur im eigens angelegten Testprofil installiert
[ ] Browser-Proxy im Testprofil zeigt auf 127.0.0.1:8080
[ ] DVWA läuft ausschließlich an 127.0.0.1 (Kapitel 3.4)
[ ] Scope ist auf DVWA begrenzt (Kapitel 8.1), bevor Intruder- oder Scan-Läufe starten
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Proxy einrichten und erste Anfrage abfangen

Ziel: Das grundlegende Setup aus Kapitel 3 praktisch nachvollziehen. **Voraussetzungen:** Burp Suite, eigenes Browser-Testprofil, DVWA läuft. **Durchführung:** Richte Proxy und Zertifikat ein (Beispiel 3.2–3.4), aktiviere Intercept und rufe DVWA im Testprofil auf (Beispiel 4.1). **Erwartetes Ergebnis:** Die Anfrage an DVWA erscheint angehalten im Intercept-Fenster. **Auswertung:** Lies die Anfrage im Detail und identifiziere Host-Header und Cookie. **Fehlerbehebung:** Erscheint nichts im Intercept-Fenster, prüfe, ob der Browser-Proxy tatsächlich im Testprofil aktiv ist und ob Intercept eingeschaltet ist.

Übung 2: HTTP History durchsuchen

Ziel: Vergangene Anfragen gezielt wiederfinden. **Voraussetzungen:** wie Übung 1, Intercept danach wieder deaktiviert. **Durchführung:** Navigiere durch mehrere DVWA-Seiten, öffne danach die HTTP History und filtere nach `id=` (Beispiel 4.5). **Erwartetes Ergebnis:** Die gefilterte Liste zeigt ausschließlich Anfragen mit diesem Parameter. **Auswertung:** Wähle eine passende Anfrage für Übung 3 aus. **Fehlerbehebung:** Leere Liste? Prüfe, ob die betreffende DVWA-Seite tatsächlich besucht wurde, während Burp Suite als Proxy aktiv war.

Übung 3: SQL-Injection systematisch im Repeater testen

Ziel: Den Repeater-Workflow aus Kapitel 5 praktisch durchlaufen. **Voraussetzungen:** DVWA-Sicherheitsstufe „Low", eine SQLi-Anfrage aus Übung 2. **Durchführung:** Sende die Anfrage an den Repeater und teste nacheinander die drei Varianten aus Beispiel 5.3. **Erwartetes Ergebnis:** Die dritte Variante (UNION SELECT) liefert Benutzername-Hash-Paare in der Antwort. **Auswertung:** Vergleiche das Ergebnis mit dem identischen Befund aus dem Ethical-Hacking-Handbuch (Kapitel 6.1 und Übung 10 dort). **Fehlerbehebung:** Kein Unterschied zwischen den Varianten erkennbar? Prüfe, ob die Sicherheitsstufe tatsächlich auf „Low" steht.

Übung 4: Login-Brute-Force mit dem Intruder

Ziel: Einen Cluster-Bomb-Angriff gegen ein Test-Login durchführen. **Voraussetzungen:** DVWA-Brute-Force-Modul, eine kleine, selbst erstellte Testwortliste. **Durchführung:** Sende eine Login-Anfrage an den Intruder, markiere das Passwortfeld als Position, wähle Angriffstyp „Sniper" und eine kleine Payload-Liste (Beispiel 6.1–6.4). **Erwartetes Ergebnis:** Ein Treffer mit abweichender Antwortlänge gegenüber den übrigen, fehlgeschlagenen Versuchen. **Auswertung:** Sortiere die Ergebnistabelle nach „Length" (Beispiel 6.5) und identifiziere den Ausreißer. **Fehlerbehebung:** Alle Antworten gleich lang? Prüfe, ob das Testpasswort tatsächlich in deiner Payload-Liste enthalten ist.

Übung 5: Base64-kodierte Zugangsdaten dekodieren

Ziel: Den Decoder praktisch einsetzen. **Voraussetzungen:** eine HTTP-Basic-Authentication-Anfrage (z. B. aus einem eigenen Testaufbau, vergleiche Wireshark-Handbuch, Übung 5). **Durchführung:** Kopiere den Base64-kodierten Wert aus dem `Authorization`-Header in den Decoder und dekodiere ihn (Beispiel 7.1). **Erwartetes Ergebnis:** Benutzername und Passwort erscheinen im Klartext. **Auswertung:** Vergleiche das Ergebnis mit der Kommandozeilen-Dekodierung aus dem Wireshark-Handbuch – beide sollten identisch sein. **Fehlerbehebung:** Unleserliches Ergebnis? Prüfe, ob tatsächlich „Base64" statt eines anderen Kodierungstyps ausgewählt wurde.

Übung 6: Zwei Antworten mit dem Comparer vergleichen

Ziel: Subtile Unterschiede zwischen zwei Antworten sichtbar machen. **Voraussetzungen:** zwei HTTP-History-Einträge, einer mit gültigem, einer mit ungültigem Login-Versuch. **Durchführung:** Markiere beide Einträge und sende sie an den Comparer (Beispiel 7.4). **Erwartetes Ergebnis:** Der Comparer hebt mindestens einen Unterschied farblich hervor. **Auswertung:** Prüfe, ob sich aus diesem Unterschied ein zuverlässiges Kriterium für „Login erfolgreich" ableiten lässt. **Fehlerbehebung:** Keine Unterschiede sichtbar? Vergewissere dich, dass die beiden gewählten Antworten tatsächlich zu einem erfolgreichen und einem fehlgeschlagenen Versuch gehören.

Übung 7: Scope definieren und Site-Map aufbauen

Ziel: Burp Suite gezielt auf DVWA begrenzen. **Voraussetzungen:** wie Übung 1. **Durchführung:** Füge DVWA zum Scope hinzu (Beispiel 8.1) und navigiere anschließend durch mehrere DVWA-Module, um die Site-Map zu füllen (Beispiel 8.2). **Erwartetes Ergebnis:** Die HTTP History zeigt nach Aktivierung von „Show only in-scope items" ausschließlich DVWA-Verkehr. **Auswertung:** Vergleiche die Site-Map-Struktur mit den dir bereits bekannten DVWA-Modulen aus dem Ethical-Hacking-Handbuch. **Fehlerbehebung:** Erscheinen weiterhin fremde Einträge, prüfe die genaue Scope-Definition (Host und Pfad) auf Tippfehler.

Übung 8: Eine Erweiterung aus dem BApp Store installieren

Ziel: Den BApp Store praktisch nutzen. **Voraussetzungen:** Internetzugang für diese eine Übung (Ausnahme vom sonst isolierten Labor, nur zum Herunterladen der Erweiterung). **Durchführung:** Installiere eine einfache, bekannte Erweiterung wie „Logger++" aus dem BApp Store (Beispiel 9.1) und wiederhole eine der vorherigen Übungen mit aktivierter Erweiterung. **Erwartetes Ergebnis:** Die Erweiterung protokolliert zusätzliche Details zum durchlaufenden Verkehr. **Auswertung:** Vergleiche den Mehrwert der Erweiterung gegenüber der Standard-HTTP-History. **Fehlerbehebung:** Erscheint die Erweiterung nicht im Menü, prüfe im Reiter „Extensions", ob die Installation tatsächlich erfolgreich abgeschlossen wurde.

Zusammenfassung

Zusammenfassung

- Dieselbe DVWA-Laborumgebung wie im Ethical-Hacking-Handbuch reicht für alle acht Übungen aus.
- Die Übungen verbinden Proxy, Repeater, Intruder, Decoder, Comparer, Scope und Erweiterungen zu einem vollständigen Workflow.
- Mehrere Übungen verweisen bewusst auf das Ethical-Hacking- und das Wireshark-Handbuch, um dieselbe Fragestellung werkzeugübergreifend zu vergleichen.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten Burp-Suite-Probleme einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
Browser zeigt „Verbindung nicht sicher"	Burp-CA-Zertifikat nicht oder falsch installiert	Zertifikat erneut herunterladen und im Testprofil als vertrauenswürdig importieren (Kapitel 3.3)
Keine Anfragen in Intercept/History sichtbar	Browser-Proxy zeigt nicht auf Burp Suite	Proxy-Einstellungen im Testprofil prüfen (Kapitel 3.2)
Intercept hält jede einzelne Anfrage an, auch irrelevante	Intercept dauerhaft aktiv, kein Scope gesetzt	Scope definieren (Kapitel 8.1) oder Intercept nur gezielt kurz aktivieren
Intruder-Lauf sehr langsam	Community-Edition-Drosselung, große Payload-Liste	kleinere Testliste verwenden, Erwartung an die Edition anpassen (Kapitel 6.4)
Decoder liefert unleserliches Ergebnis	falscher Kodierungstyp ausgewählt	andere Kodierung ausprobieren (Base64, URL, Hex)
Scanner-Funktion nicht verfügbar	Community statt Professional Edition	Scanner ist ausschließlich Teil der Professional Edition (Kapitel 1.2)

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten Burp-Suite-Arbeitsschritte auf einen Blick

```
# Grundsetup
Proxy-Listener pruefen: 127.0.0.1:8080
Browser-Testprofil mit diesem Proxy und dem Burp-CA-Zertifikat einrichten

# Verkehr einsehen
Proxy -> Intercept       (einzelne Anfrage anhalten)
Proxy -> HTTP history    (vollstaendiger Mitschnitt)
# Gezielt testen
Rechtsklick -> "Send to Repeater"   (einzelne Anfrage wiederholt senden)
Rechtsklick -> "Send to Intruder"   (automatisierte Variation)
Rechtsklick -> "Send to Comparer"   (zwei Antworten gegenueberstellen)

# Scope und Struktur
Target -> Site map -> "Add to scope"
Proxy -> Options -> "Show only in-scope items"
# Decoder
Decoder -> Text eingeben -> "Decode as" -> Base64 / URL / HTML / Hex
```

11.3 Glossar

- **BApp Store** – das in Burp Suite integrierte Verzeichnis offizieller und community-entwickelter Erweiterungen.
- **Comparer** – Werkzeug zum Gegenüberstellen zweier Texte oder HTTP-Antworten.
- **Decoder** – Werkzeug zum Umwandeln gängiger Web-Kodierungen in beide Richtungen.
- **Intercept** – Funktion, die einzelne Anfragen vor dem Senden anhält und zur Bearbeitung anzeigt.
- **Intruder** – Werkzeug zur automatisierten Variation von Anfragen über Positionsmarker und Payload-Listen.
- **Proxy** – ein zwischengeschalteter Vermittler, der Datenverkehr einsehen und verändern kann.
- **Repeater** – Werkzeug zum gezielten, wiederholten Senden einer einzelnen, veränderbaren Anfrage.
- **Scope** – die bewusst festgelegte Begrenzung von Burp Suite auf eine bestimmte Anwendung.
- **Site Map** – die sich automatisch aufbauende Baumstruktur aller beobachteten Pfade und Parameter einer Anwendung.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
PortSwigger Web Security Academy	kostenlose, sehr detaillierte Lernplattform zu Web-Schwachstellen mit eigenen Übungslaboren
Offizielle Burp-Suite-Dokumentation (portswigger.net/burp/documentation)	vollständige, verbindliche Referenz
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: SQL-Injection, XSS, CSRF, Reporting
Wireshark-Handbuch (dieselbe Buchreihe)	Netzwerkseite der Protokollanalyse, ergänzend zur Anwendungsschicht
TryHackMe / HackTheBox	legale Übungsplattformen mit eigenen Web-Application-Räumen

TIPP – Profi-Trick

Die PortSwigger Web Security Academy bietet zu praktisch jeder im Ethical-Hacking-Handbuch behandelten Web-Schwachstelle eine eigene, kostenlose Lernumgebung mit direktem Browserzugriff, ganz ohne eigenes Labor – eine hervorragende Ergänzung zu den Übungen in diesem Buch.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen Zertifikatsinstallation, Proxy-Konfiguration und die Grenzen der Community Edition.
- Der Spickzettel deckt Grundsetup, Verkehrseinsicht, gezieltes Testen, Scope und Decoder ab.
- Die PortSwigger Web Security Academy, die offizielle Dokumentation sowie das Ethical-Hacking- und Wireshark-Handbuch sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Die wichtigsten Burp-Suite-Tabs (Konzept) … Seite 8
Beispiel 2.1: Normale Verbindung vs. Verbindung über einen Proxy (Konzept) … Seite 9
Beispiel 2.2: Das Grundproblem (Konzept) … Seite 9
Beispiel 2.3: Zwei getrennte TLS-Verbindungen (Konzept) … Seite 9
Beispiel 3.1: Burp Suite unter Kali starten … Seite 11
Beispiel 3.2: Proxy-Listener in Burp Suite prüfen … Seite 11
Beispiel 3.3: Browser-Proxy auf Burp Suite umstellen (Konzept, Firefox) … Seite 11
Beispiel 3.4: Zertifikat herunterladen und installieren (Konzept) … Seite 11
Beispiel 3.5: DVWA starten (Wiederholung aus dem Ethical-Hacking-Handbuch) … Seite 12
Beispiel 4.1: Intercept aktivieren und eine Anfrage abfangen … Seite 13
Beispiel 4.2: Eine abgefangene Anfrage anzeigen und bearbeiten … Seite 13
Beispiel 4.3: HTTP History durchsuchen … Seite 13
Beispiel 4.4: Einen Eintrag der History im Detail ansehen … Seite 13
Beispiel 4.5: HTTP History nach Methode und Statuscode filtern … Seite 14
Beispiel 5.1: Eine Anfrage aus der History an den Repeater senden … Seite 15
Beispiel 5.2: Ursprüngliche Anfrage im Repeater … Seite 15
Beispiel 5.3: Denselben Parameter systematisch verändern und senden … Seite 15
Beispiel 5.4: Der Repeater-Verlauf … Seite 16
Beispiel 5.5: HTTP-Methode direkt im Repeater ändern … Seite 16
Beispiel 6.1: Eine Anfrage an den Intruder senden und Position markieren … Seite 17
Beispiel 6.2: Eine markierte Anfrage (Konzept) … Seite 17
Beispiel 6.3: Cluster Bomb gegen Benutzername und Passwort (Konzept) … Seite 18
Beispiel 6.4: Eine einfache Payload-Liste einsetzen … Seite 18
Beispiel 6.5: Ergebnisse nach Antwortlänge sortieren … Seite 18
Beispiel 7.1: Base64 dekodieren … Seite 20
Beispiel 7.2: URL-Kodierung umwandeln … Seite 20
Beispiel 7.3: Mehrfach kodierte Werte entschlüsseln … Seite 21
Beispiel 7.4: Zwei Antworten vergleichen … Seite 21
Beispiel 8.1: Scope auf DVWA begrenzen … Seite 22
Beispiel 8.2: Struktur einer Anwendung erkunden … Seite 22
Beispiel 8.3: Einen Scan starten (Konzept, nur Professional Edition) … Seite 22
Beispiel 9.1: BApp Store öffnen und eine Erweiterung installieren … Seite 24
Checkliste 10.1: Vor jeder Übung … Seite 26
Beispiel 11.1: Die wichtigsten Burp-Suite-Arbeitsschritte auf einen Blick … Seite 29

Index

BApp Store – 24, 28, 30
Base64 – 20, 27
Burp Suite – 5, 7, 8, 9, 11, 20, 22, 24
Burp-CA-Zertifikat – 9, 11
Cluster Bomb – 18
Comparer – 5, 7, 21, 27, 30
Decoder – 5, 7, 20, 21, 24, 27, 30
HTTP History – 13, 14, 26, 27
Intercept – 13, 26, 30
Intruder – 5, 7, 8, 17, 27, 30
PortSwigger – 11
Positionsmarker – 30
Proxy – 5, 7, 8, 9, 11, 15, 22, 26
Repeater – 5, 7, 8, 15, 16, 21, 27, 30
Scanner – 5, 7, 22, 23
Scope – 5, 22, 27, 30
Site Map – 30
Sniper – 27

BURP SUITE DAS PRAXISHANDBUCH

Was zwischen Browser und Server wirklich passiert

Burp Suite schaltet sich als Proxy zwischen Browser und Webanwendung und macht jede HTTP-Anfrage sichtbar, veränderbar und wiederholbar. Dieses Handbuch vertieft, was im Ethical-Hacking-Handbuch nur kurz angerissen wurde: Proxy und Repeater für systematisches Testen, Intruder für automatisierte Variation, Decoder und Comparer als Hilfswerkzeuge, sowie Scope und Scanner für den professionellen Workflow.

DAS ERWARTET DICH

- HTTP-Proxy-Prinzip und TLS-Interception verständlich erklärt
- Proxy und Repeater: SQL-Injection und XSS systematisch statt zufällig finden
- Intruder: Sniper, Battering Ram, Pitchfork und Cluster Bomb im Detail
- Decoder und Comparer als Hilfswerkzeuge für Kodierung und Antwortvergleich
- Scope, Scanner, BApp Store, 8 geführte Übungen gegen DVWA und Glossar

Für wen?

Leser:innen des Ethical-Hacking-Handbuchs und der anderen Bände dieser Reihe, die Web-Application-Testing vertiefen möchten, sowie Studierende und Auszubildende. Ausschließlich für legale, autorisierte Tests und eigene Laborumgebungen.

32 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking-Handbuch und der gesamten Reihe

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.